MovieLens Fusion: An Integrated Movie Analytics Pipeline

A Project Report Submitted in Partial Fulfillment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

2420030001 – DIKSHA YADAV
2420030332 – K. BHAVANA
2420030430 – G. TEJASHREE
2420030471 – N. NIKITHA REDDY

Under the Esteemed Guidance of

Ms. B. PRATHUSHA
Assistant Professor
Department of Computer Science and Engineering
KLH CSE, Bowrampet Campus
Academic Year: 2026–2027

Semester: Odd

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "MovieLens Fusion: An Integrated Movie Analytics Pipeline" is a record of bonafide work carried out by DIKSHA YADAV – 2420030001, K. BHAVANA – 2420030332, G. TEJASHREE – 2420030430, and N. NIKITHA REDDY – 2420030471, submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering to K L (Deemed to be) University. The results embodied in this report have not been copied from any other department, university, or institute.

DIKSHA YADAV – 2420030001

K. BHAVANA – 2420030332

G. TEJASHREE – 2420030430

N. NIKITHA REDDY – 2420030471

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project report entitled "MovieLens Fusion: An Integrated Movie Analytics Pipeline" is a bonafide work carried out and submitted by DIKSHA YADAV – 2420030001, K. BHAVANA – 2420030332, G. TEJASHREE – 2420030430, and N. NIKITHA REDDY – 2420030471, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Guide	Course Coordinator	Head of the Department
_____	_____	_____
Ms. B. PRATHUSHA	Ms. Nalajala Sunanda	Dr. P. Venkateswara Rao
Assistant Professor, CSE	Assistant Professor, CSE	Department of CSE

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely help, support, and guidance of many people. We would like to express our sincere thanks to everyone who has assisted us in one way or another throughout the completion of our project.

We express our deepest gratitude to our Course Coordinator and Project Guide, Ms. B. PRATHYUSHA, Assistant Professor, Department of Computer Science and Engineering, for her valuable guidance, continuous encouragement, support, and suggestions throughout the development of our project, "MovieLens Fusion: An Integrated Movie Analytics Pipeline."

We extend our sincere gratitude to Dr. P. Venkateswara Rao, Head of the Department of Computer Science and Engineering, for providing us with the necessary facilities, resources, and support to complete this project successfully.

We also thank all the teaching and non-teaching staff members of the Department of Computer Science and Engineering for their assistance and cooperation. Finally, we express our heartfelt thanks to our parents, friends, and classmates for their constant encouragement, valuable suggestions, and support throughout the project.

DIKSHA YADAV – 2420030001

K. BHAVANA – 2420030332

G. TEJASHREE – 2420030430

N. NIKITHA REDDY – 2420030471

ABSTRACT

Movie data analytics plays an important role in understanding movie trends, genres and content distribution. However, raw movie datasets may contain missing values, inconsistent formats, and unstructured information, making them difficult to analyze directly.

The project "MovieLens Fusion: An Integrated Movie Analytics Pipeline" focuses on developing an end-to-end data engineering and analytics platform for processing movie and TV show information. The system uses a dataset containing 8,807 records, including 6,131 movies and 2,676 TV shows. It aims to transform raw data into a clean, structured, and analysis-ready format.

The project uses Python and Pandas for data inspection and cleaning, followed by PySpark for data transformation and processing. Operations such as handling missing values, removing duplicates, standardizing data, and extracting genre information are performed to improve data quality. The processed data is stored in Parquet format and organized in a MySQL database using relational tables such as movies, genres, and movie_genres.

A web-based application is developed using React for the frontend and Flask for the backend to provide an interface for accessing and exploring movie and TV show information. The application connects the data processing and database layers to support interactive data exploration and analytics. Docker is used to containerize the frontend and backend components for easier deployment.

The project demonstrates how data engineering technologies can be combined to build a structured movie analytics pipeline. It provides practical experience in data cleaning, transformation, database management, web application development, and containerization.

Keywords: Movie Analytics, Data Engineering, Python, Pandas, PySpark, MySQL, Parquet, Flask, React, Docker.

Index

S. No.	Chapters	Topics	Page No.
		Acknowledgement	3
		Abstract	5
1	Introduction	1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	2
		1.4 Objectives	2
		1.5 Importance of the Application	2
		1.6 Target Users/Audience	3
		1.7 Literature Survey	3
		1.8 Problem Gap and Proposed Approach	4
2	System Requirements	2.1 Hardware Requirements	6
		2.2 Software Requirements	6
		2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Data Processing – Python and Pandas	8
		3.2 Big Data Processing – PySpark	8
		3.3 Database Management – MySQL	8
		3.4 Data Storage – CSV and Parquet	8
		3.5 Backend – Flask	9
		3.6 Frontend – React and Vite	9
		3.7 Containerization – Docker	9
4	System Architecture	4.1 High-Level Architecture Diagram	10
		4.2 Description of System Modules	10
		4.3 Application Execution Architecture	12
5	Design	5.1 User Interface Design	13
		5.2 System Workflow / Data Flow Diagram	13
		5.3 Database Design	16
		5.4 Frontend and Backend Communication	17
6	Implementation	6.1 Module-wise Implementation	18
		6.2 Data Ingestion	18
		6.3 Data Cleaning and Preprocessing	18
		6.4 Data Transformation Using PySpark	19
		6.5 Genre Extraction and Processing	19
		6.6 MySQL Database Implementation	20
		6.7 Backend Implementation Using Flask	20
		6.8 Frontend Implementation Using React	21
7	Features	7.1 List of Main Features	22

S. No.	Chapters	Topics	Page No.
		7.2 Data Exploration and Search	23
		7.3 Movie and TV Show Analysis	23
		7.4 Genre-based Analysis	24
		7.5 Interactive Analytics Dashboard	25
8	Testing	8.1 Functional Testing	26
		8.2 Integration Testing	26
		8.3 Database Testing	26
		8.4 Test Cases and Results	27
		8.5 Data Quality Evaluation	27
		8.6 Application and Deployment Verification	28
9	Deployment	9.1 Docker Configuration	29
		9.2 Backend Containerization	30
		9.3 Frontend Containerization	30
		9.4 Docker Compose Execution	30
		9.5 Local Application Execution	31
10	Challenges and Limitations	10.1 Issues Faced During Development	32
		10.2 Solutions Applied	32
		10.3 Current Limitations	33
11	Future Enhancements	11.1 Multi-source Data Integration	35
		11.2 Advanced Recommendation Systems	35
		11.3 User Preference-based Recommendations	35
		11.4 Cloud Deployment	35
12	Conclusion	12.1 Summary of the Project	36
		12.2 Achievements	36
		12.3 Skills Learned During Development	37
13	References	Research Papers	38
		Dataset References	38
		Technical Documentation	38
14	Appendices	14.1 Project Components and Configuration Files	39
		14.2 Installation and Setup Instructions	41
		14.3 User Manual	43
		14.4 Team Contributions	43

CHAPTER 1
Introduction

1.1 Background of the Project

Movies and television shows are among the most widely consumed forms of digital entertainment. Streaming platforms and public movie datasets describe every title using attributes such as type, director, cast, country, release year, rating, duration and genre. When these records are analyzed properly, they reveal useful patterns about content distribution, popular genres and the composition of a catalogue.

Raw movie datasets, however, are rarely ready for analysis. Some fields are empty, some values follow inconsistent formats, and some columns store several values inside a single cell. For example, the duration field mixes numbers with units, and the genre field (listed_in) can contain more than one genre for a single title. Such data must be inspected, cleaned and restructured before it can support reliable queries and visualizations.

MovieLens Fusion is an end-to-end data engineering and analytics project that addresses this need. The project takes a movie and TV show CSV dataset containing 8,807 records (6,131 movies and 2,676 TV shows) and moves it through a complete pipeline: Python and Pandas for inspection and cleaning, PySpark for transformation, Parquet files for processed storage, and a MySQL database named movielens_fusion for relational storage. A Flask backend exposes the stored data through API endpoints, and a React and Vite frontend presents it to users. The backend and frontend are packaged with Docker for deployment.

1.2 Problem Statement

Raw movie and TV show data is difficult to analyze directly because of missing values, mixed data formats and multi-valued columns. Manually exploring a CSV file with thousands of rows is slow, and a flat file cannot easily answer questions such as which titles belong to a particular genre or how movies and TV shows are distributed.

The problem addressed in this project is to design and build an integrated pipeline that ingests the raw dataset, improves its quality, transforms it into a structured form, stores it in a relational database, and makes it accessible through a web application for exploration and analytics.

1.3 Scope of the Project

The scope of the project includes the following activities:

1. Reading and inspecting a movie and TV show CSV dataset of 8,807 records.
2. Cleaning the data using Python and Pandas, including missing value handling, duplicate checking and standardization.
3. Transforming the data with PySpark, including duration processing and genre extraction.
4. Storing the processed data in Parquet format.
5. Organizing the data in a MySQL database with the tables movies, genres and movie_genres.
6. Developing a Flask backend that serves data from MySQL through API endpoints.
7. Developing a React and Vite frontend for data exploration, search and analytics.
8. Containerizing the frontend and backend using Docker and Docker Compose.

The following items are outside the scope of the current implementation and are discussed only as future enhancements in Chapter 11: integration of additional movie datasets, advanced recommendation systems, user preference-based recommendations and cloud deployment.

1.4 Objectives

The main objectives of the project are:

1. To build a data engineering pipeline that converts a raw movie and TV show CSV file into analysis-ready data.
2. To inspect and clean the dataset and report the data quality results.
3. To use PySpark for scalable data transformation and genre extraction.
4. To store processed data in Parquet format and load it into a normalized MySQL schema.
5. To model the many-to-many relationship between movies and genres using a junction table.
6. To develop a web application with a Flask backend and a React frontend for interactive exploration of the data.
7. To package the application using Docker for simpler deployment.

1.5 Importance of the Application

The application demonstrates how separate data engineering technologies can be combined into one working pipeline. Cleaning, transformation, storage, an API layer and a user interface

are usually studied separately; this project connects them so that raw data can be traced all the way to what a user sees on screen.

The relational design also makes the data easier to query. By separating genres into their own table, a title with several genres is represented correctly, and genre-based filtering and counting become straightforward database operations rather than text searches over a column. The Docker packaging shows how the application can be started in a repeatable way on another machine.

1.6 Target Users/Audience

The intended users of the application and this report include:

- Students learning data engineering, big data processing and full-stack web development.
- Faculty members and evaluators reviewing the project.
- Developers who want a reference for connecting Pandas, PySpark, MySQL, Flask and React.
- Users who wish to explore movie and TV show information and genre trends.

1.7 Literature Survey

Several research works address the individual layers on which this project is built: distributed data processing, in-memory data analysis, data quality, relational storage and movie datasets. Table 1.1 summarizes the works that were reviewed.

Table 1.1: Summary of the literature reviewed

Ref.	Authors and Title	Methodology	Findings	Limitation with Respect to This Project
[1]	M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," Communications of the ACM, 2016.	Presents Spark as one engine for batch, SQL, streaming, machine-learning and graph workloads, built on the resilient distributed dataset abstraction.	Shows that a single engine can support several processing models, which simplifies the construction of data pipelines.	Concerned with the processing engine; it does not cover relational storage or a user-facing application.
[2]	M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," NSDI, 2012.	Introduces resilient distributed datasets, an in-memory abstraction with lineage-based fault recovery, and implements it in Spark.	Reports that Spark performs better than Hadoop for iterative and interactive workloads.	Targets large clusters; the dataset in this project is small enough to be held on one machine.

Ref.	Authors and Title	Methodology	Findings	Limitation with Respect to This Project
[3]	W. McKinney, "Data Structures for Statistical Computing in Python," Proc. 9th Python in Science Conference, 2010.	Describes the DataFrame and related data structures of the pandas library.	Shows that labeled tabular structures make loading, cleaning and manipulating data concise, including the handling of missing data.	Works on in-memory data on a single machine, so it suits the cleaning stage but not very large data.
[4]	E. Rahm and H. H. Do, "Data Cleaning: Problems and Current Approaches," IEEE Data Engineering Bulletin, 2000.	Surveys data quality problems, classified as single-source or multi-source and as schema-level or instance-level, and reviews cleaning approaches.	Identifies the main cleaning activities: analyzing the data, transforming it and verifying the result.	A general survey; it does not describe an implementation for movie data.
[5]	R. Y. Wang and D. M. Strong, "Beyond Accuracy: What Data Quality Means to Data Consumers," Journal of Management Information Systems, 1996.	Uses a survey of data consumers to identify the attributes that define data quality.	Produces a framework of data quality dimensions grouped into intrinsic, contextual, representational and accessibility categories.	A conceptual framework; it does not provide a processing pipeline.
[6]	E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," Communications of the ACM, 1970.	Proposes the relational model, in which data is organized as relations, and discusses removing redundancy.	Provides the basis for tables, keys and joins in relational databases.	A theoretical foundation; it does not address web applications or analytics.
[7]	F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," ACM Transactions on Interactive Intelligent Systems, 2015.	Describes the history, structure and uses of the MovieLens datasets collected by the GroupLens research group.	Documents how movie rating datasets have been used in recommender-system research.	Centred on user rating data; the dataset used in this project is a catalogue of titles and contains no user ratings.

1.8 Problem Gap and Proposed Approach

The reviewed works describe the layers of a data-driven application separately: distributed processing [1], [2], in-memory data analysis [3], data quality [4], [5], relational storage [6] and movie datasets [7]. Taken alone, none of them shows how these layers are combined step by step to take a raw movie and TV show CSV file to an application that a user can explore. Three gaps follow from this:

- The processing, data quality and storage techniques are described independently of each other and of the application that finally uses the data.
- Catalogue data contains multi-valued fields such as genre. Such fields have to be restructured before they can be queried relationally, and this restructuring is not covered by the general works.

- The data produced by a pipeline is rarely packaged together with its API and user interface so that the whole system can be started in a repeatable way.

MovieLens Fusion addresses these gaps by implementing the complete chain in one project. Pandas inspects and cleans the data, PySpark transforms it and extracts the genres, Parquet files hold the processed data, and a normalized MySQL schema with a many-to-many relationship stores it. A Flask backend serves the data, a React and Vite frontend presents it, and Docker Compose packages the backend and frontend for repeatable deployment.

CHAPTER 2
System Requirements

2.1 Hardware Requirements

The project does not depend on specialized hardware. Table 2.1 lists a suggested configuration for running the data pipeline, the database and the Docker containers on one machine. These values are general recommendations for this software stack.

Table 2.1: Suggested hardware configuration

Component	Minimum	Recommended
Processor	Dual-core processor (Intel Core i3 or equivalent)	Quad-core processor (Intel Core i5 or equivalent, or better)
RAM	8 GB	16 GB (PySpark and Docker run more comfortably)
Storage	5 GB free disk space	10 GB or more free disk space
Display	Standard laptop or desktop display	Full HD display
Network	Required to download packages and Docker images	Stable broadband connection
GPU	Not required	Not required

2.2 Software Requirements

The software required to build and run the project is listed below.

- Operating system: Windows 10/11. The MySQL server runs directly on the Windows host machine. The Docker containers themselves do not depend on the host operating system.
- Python 3.x with pip and virtual environment support.
- Java Development Kit (JDK), which is required by Apache Spark and therefore by PySpark.
- MySQL Server, hosting the database movielens_fusion.
- Node.js and npm, used to install frontend dependencies and to run or build the React and Vite application.
- Docker and Docker Compose, used to containerize and run the frontend and backend.
- A modern web browser such as Google Chrome, Microsoft Edge or Mozilla Firefox.

2.3 Development Tools and Frameworks

Table 2.2: Development tools and frameworks

Tool / Framework	Category	Purpose in the Project
Python	Programming language	Data cleaning scripts, PySpark jobs and the Flask backend
Pandas	Data analysis library	Reading the CSV file, inspection and cleaning of the dataset
PySpark	Big data processing	Transformation, genre extraction, aggregation and writing Parquet files
Parquet	Columnar file format	Storage of the processed data
MySQL	Relational database	Tables movies, genres and movie_genres in movielens_fusion
Flask	Backend web framework	API endpoints between the database and the frontend
Gunicorn	WSGI server	Runs the Flask application inside the backend container
React	Frontend library	Component-based user interface
Vite	Frontend build tool	Development server and production build of the React application
Nginx	Web server	Serves the built frontend inside the frontend container
Docker / Docker Compose	Containerization	Packaging and running the frontend and backend containers

CHAPTER 3
Technology Stack

3.1 Data Processing – Python and Pandas

Python [9] is the main programming language of the project. It is used for the cleaning scripts, the PySpark jobs and the Flask backend. Pandas is a Python library that provides the DataFrame, a table-like structure with labeled columns [3], [10].

In this project Pandas performs the first stage of the pipeline. It reads the raw CSV file, inspects the structure and data types, identifies missing values, checks for duplicate records, standardizes values and converts data types where required. Pandas is well suited to this stage because the dataset (8,807 rows) fits easily in memory and the library offers simple operations for inspection and cleaning.

3.2 Big Data Processing – PySpark

Apache Spark is a distributed data processing engine, and PySpark is its Python interface [1, 11]. PySpark provides a DataFrame API with schema inspection, column selection, filtering, grouping, aggregation and built-in functions for working with text columns.

In MovieLens Fusion, PySpark reads the processed data, inspects its schema, selects and filters columns, transforms duration information, extracts genre information, groups and aggregates records, and writes the result to Parquet files. Although the dataset is modest in size, using PySpark makes the transformation stage follow the same pattern that would be needed for much larger data.

3.3 Database Management – MySQL

MySQL is an open-source relational database management system. The project uses a database named movielens_fusion with three tables: movies (8,807 records), genres (42 records) and movie_genres (19,323 records). Relational storage enforces a clear structure, allows joins across tables, and supports the filtering and counting queries required by the backend [6, 13].

3.4 Data Storage – CSV and Parquet

The raw dataset is supplied as a CSV file, a plain-text format that is easy to read but stores no data types and no compression. Parquet [12] is a columnar storage format that keeps the schema together with the data and is designed for efficient analytical processing. In the pipeline the

CSV file is the input, and the processed output of PySpark is written as Parquet files before the data is loaded into MySQL.

3.5 Backend – Flask

Flask [14] is a lightweight Python web framework. It is used to build the backend of the application, which receives HTTP requests from the frontend, runs queries against the MySQL database and returns the results as JSON responses. The application registers its routes as separate modules (Flask Blueprints), enables Cross-Origin Resource Sharing (CORS) so that the browser-based frontend can call the API, and provides a health-check endpoint for verifying the service. In the Docker deployment the Flask application is served by Gunicorn [19] and listens on port 5000 inside the backend container.

3.6 Frontend – React and Vite

React [15] is a JavaScript library for building user interfaces from reusable components. Vite [16] is a frontend build tool that provides a fast development server and produces an optimized production build. The frontend of MovieLens Fusion is written in JavaScript and CSS using React and Vite, and communicates with the Flask backend through API calls.

3.7 Containerization – Docker

Docker [17] packages an application and its dependencies into a container so that it runs the same way on different machines. Docker Compose [18] describes and starts several containers together. The project defines two containers: a backend container based on Python and Gunicorn, and a frontend container that uses Node.js to build the application and Nginx [20] to serve it on port 3000. MySQL is not part of the Compose configuration; the backend connects to the existing MySQL database running on the Windows host machine.

CHAPTER 4
System Architecture

4.1 High-Level Architecture Diagram

Figure 4.1 shows the complete architecture of MovieLens Fusion. The raw CSV dataset is inspected and cleaned with Python and Pandas, transformed with PySpark and written to Parquet storage. The processed data is loaded into the MySQL database, which the Flask backend queries. The React frontend retrieves data from the backend and presents it to the user. The backend and frontend run as Docker containers, while MySQL runs on the host machine.

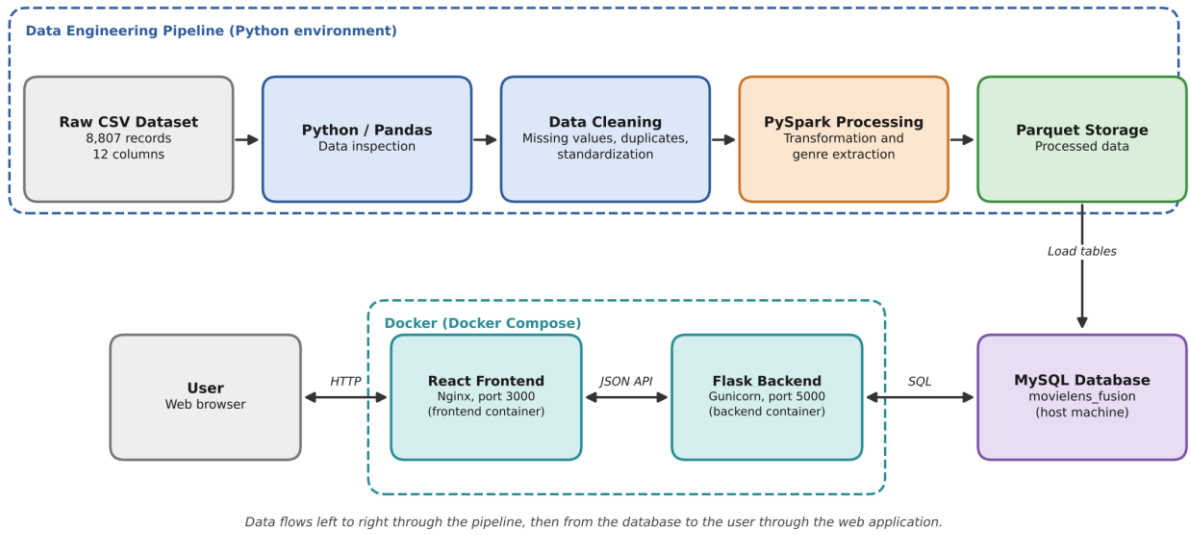


Figure 4.1: High-Level System Architecture of MovieLens Fusion

The architecture can be divided into three parts. The first is the data engineering pipeline, which runs once (or whenever the dataset is refreshed) and prepares the data. The second is the storage layer, consisting of Parquet files and the MySQL database. The third is the web application, which is used interactively by the user and is the part deployed with Docker.

4.2 Description of System Modules

The system is organized into seven modules, shown in Figure 4.2 and summarized in Table 4.1.

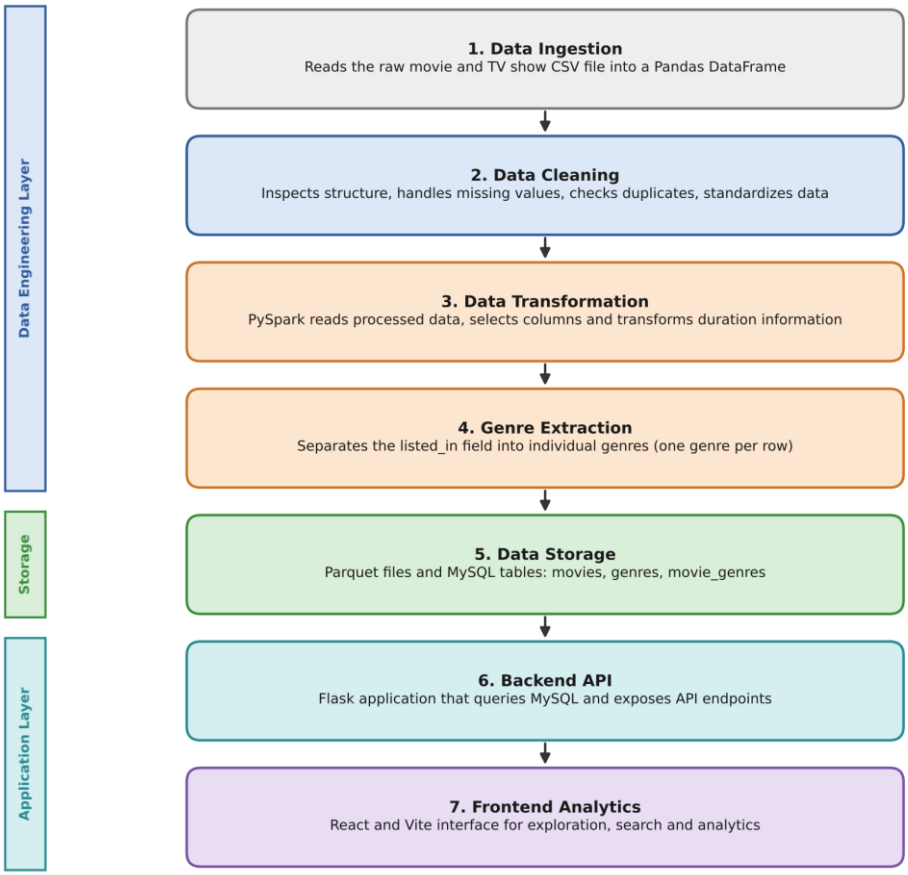


Figure 4.2: System Modules of MovieLens Fusion

Table 4.1: Summary of system modules

Module	Technology	Input	Output
1. Data Ingestion	Python, Pandas	Raw CSV file (8,807 records, 12 columns)	DataFrame in memory
2. Data Cleaning	Python, Pandas	Raw DataFrame	Cleaned dataset (14 columns) and cleaning report
3. Data Transformation	PySpark	Cleaned data	Transformed DataFrame (duration fields, selected columns)
4. Genre Extraction	PySpark	listed_in values	Individual genre values linked to each title
5. Data Storage	Parquet, MySQL	Processed data	Parquet files; tables movies, genres, movie_genres
6. Backend API	Flask, Python	HTTP requests from the frontend	JSON responses built from MySQL queries
7. Frontend Analytics	React, Vite, JavaScript, CSS	API responses	Pages, search results and visualizations for the user

4.3 Application Execution Architecture

Figure 4.3 shows how the application runs once the data has been loaded. The user opens the React frontend in a web browser. When the user performs an action, the frontend sends a request to a Flask API endpoint. Flask runs the corresponding query on the MySQL database, receives the result rows, converts them into a response and returns it to the frontend, which updates the page.

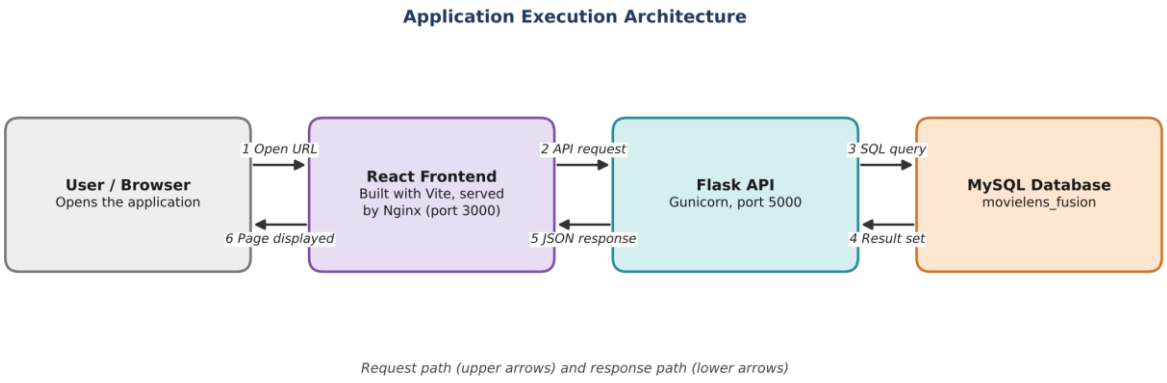


Figure 4.3: Application Execution Architecture

In the Docker deployment the frontend is served by Nginx on port 3000, and the Flask application runs under Gunicorn on port 5000 inside the backend container. The backend reaches the MySQL database that is already running on the host machine. This arrangement keeps the web application portable while the database remains outside the Compose configuration.

CHAPTER 5
Design

5.1 User Interface Design

The user interface is built with React, Vite, JavaScript and CSS. It is designed so that a user can move from an overview of the data to specific titles and genres without writing any queries. The interface provides the following views, which are described in Chapter 7:

- a dashboard that summarizes the dataset;
- search and filtering of movies and TV shows;
- information about individual movies and TV shows;
- genre-based analysis;
- analytics charts.

5.2 System Workflow / Data Flow Diagram

Figure 5.1 shows the data processing workflow. Data enters as a raw CSV file and passes through nine ordered steps before it is available in MySQL. Steps 2 to 5 are carried out with Python and Pandas, steps 6 and 7 with PySpark, and steps 8 and 9 write the processed data to storage.

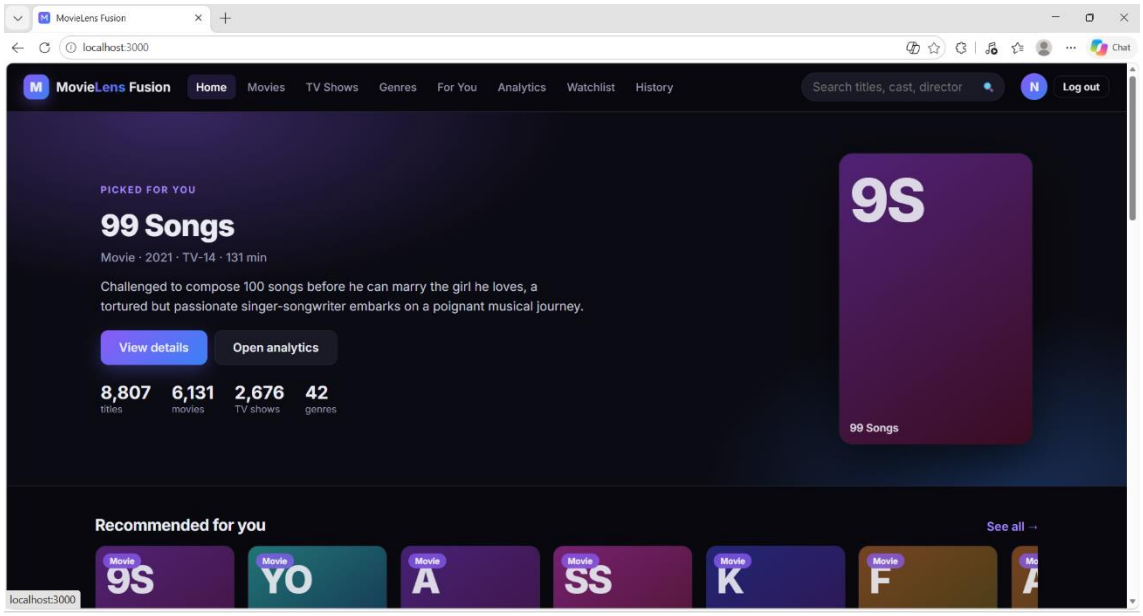


Figure 5.1: MovieLens Fusion Home Page

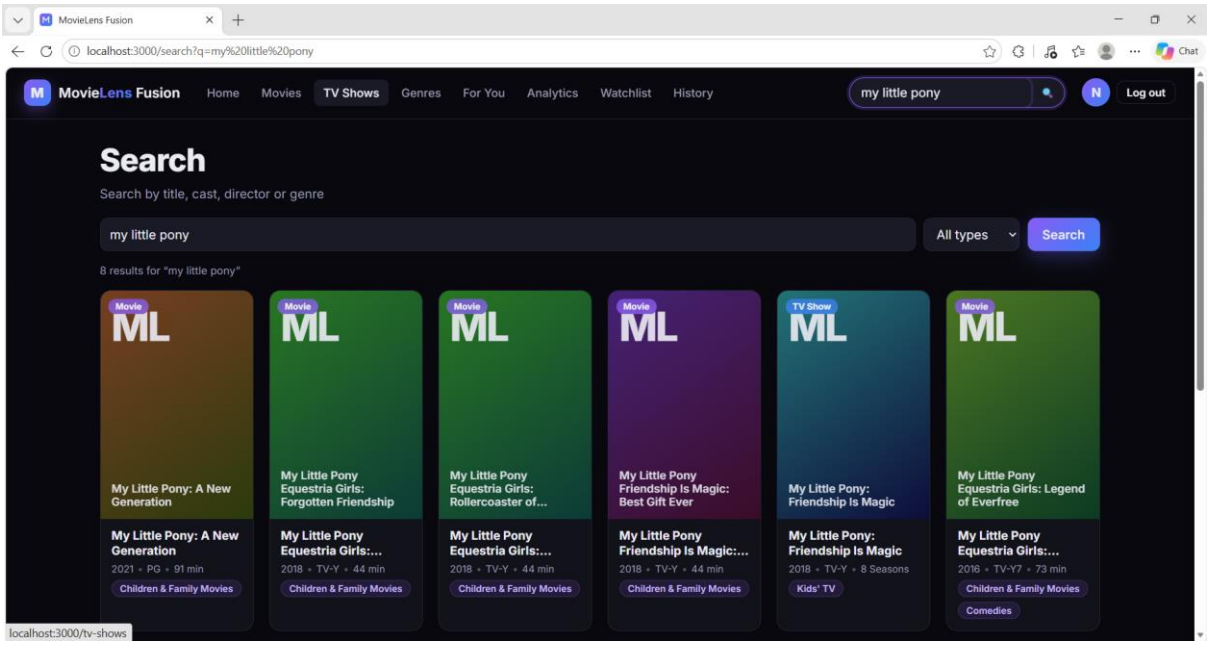


Figure 5.2: Search and Filtering Interface

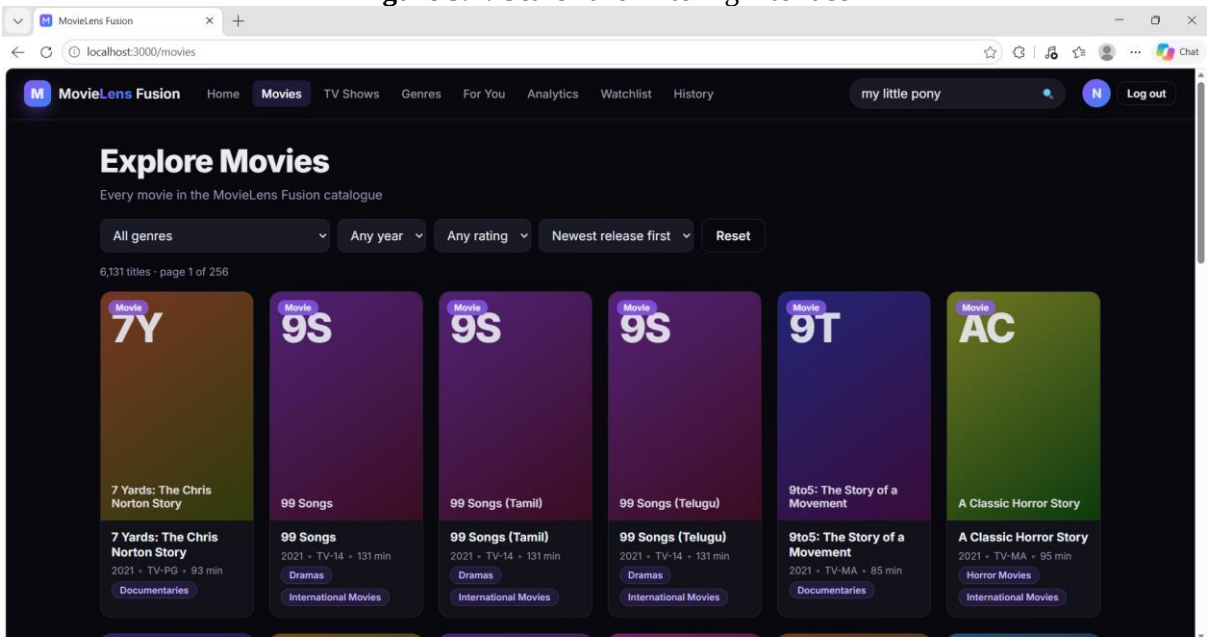


Figure 5.3: Movie and TV Show Exploration Page

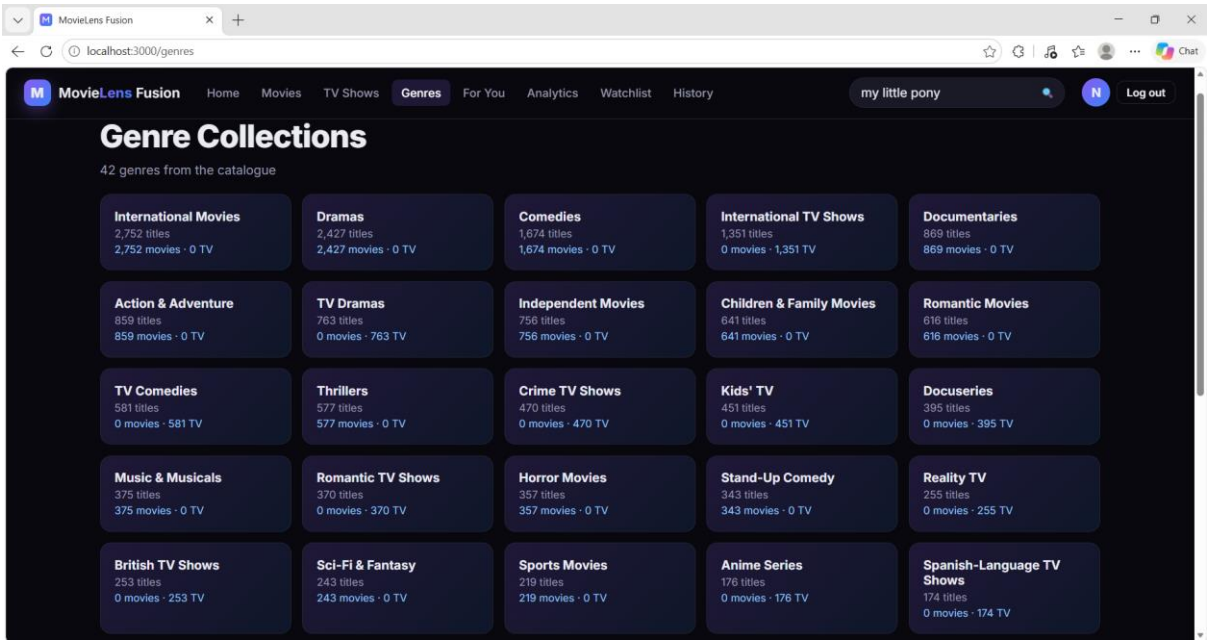


Figure 5.4: Genre Analytics View

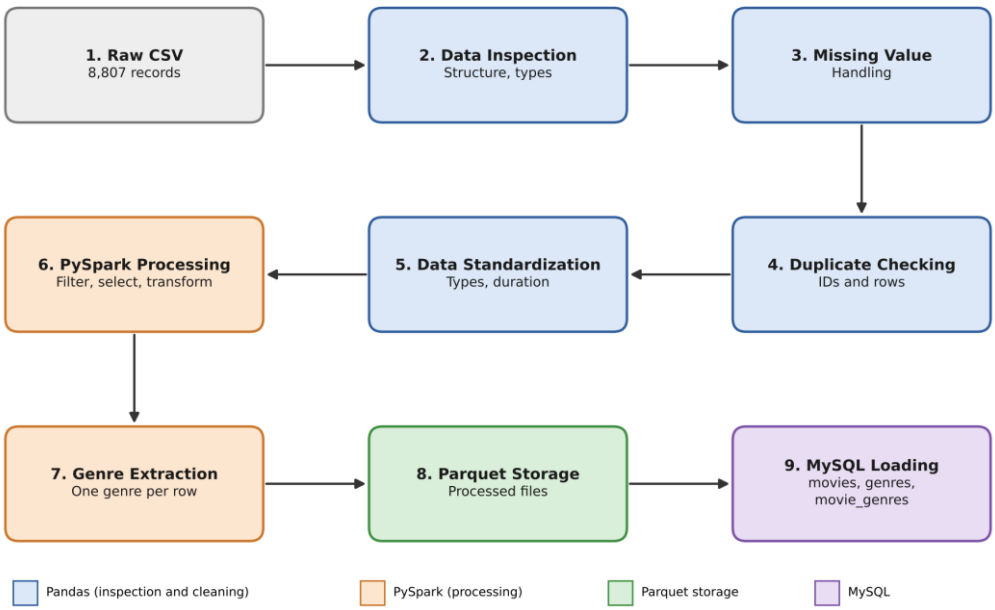


Figure 5.5: Data Processing Workflow

Table 5.1: Data processing steps

Step	Operation	Tool	Result
1	Read raw CSV	Pandas	8,807 records loaded
2	Data inspection	Pandas	Structure, data types and column details reviewed
3	Missing value handling	Pandas	Missing values identified and handled
4	Duplicate checking	Pandas	Duplicate IDs: 0; duplicate rows: 0

Step	Operation	Tool	Result
5	Data standardization	Pandas	Cleaned dataset with 14 columns; rejected records: 0
6	PySpark processing	PySpark	Schema inspection, selection, filtering, duration transformation
7	Genre extraction	PySpark	Genre values separated for each title
8	Parquet storage	PySpark	Processed data written to Parquet files
9	MySQL loading	MySQL	Tables movies, genres and movie_genres populated

5.3 Database Design

Figure 5.2 shows the relational design of the movielens_fusion database. The movies table holds one row per title and uses show_id as its primary key. The genres table holds one row per distinct genre (42 rows) with genre_id as its primary key. A title can belong to several genres and a genre can contain many titles, which is a many-to-many relationship. This relationship is resolved by the movie_genres table, which stores pairs of show_id and genre_id (19,323 rows). Each of its two columns is a foreign key: show_id refers to movies and genre_id refers to genres.

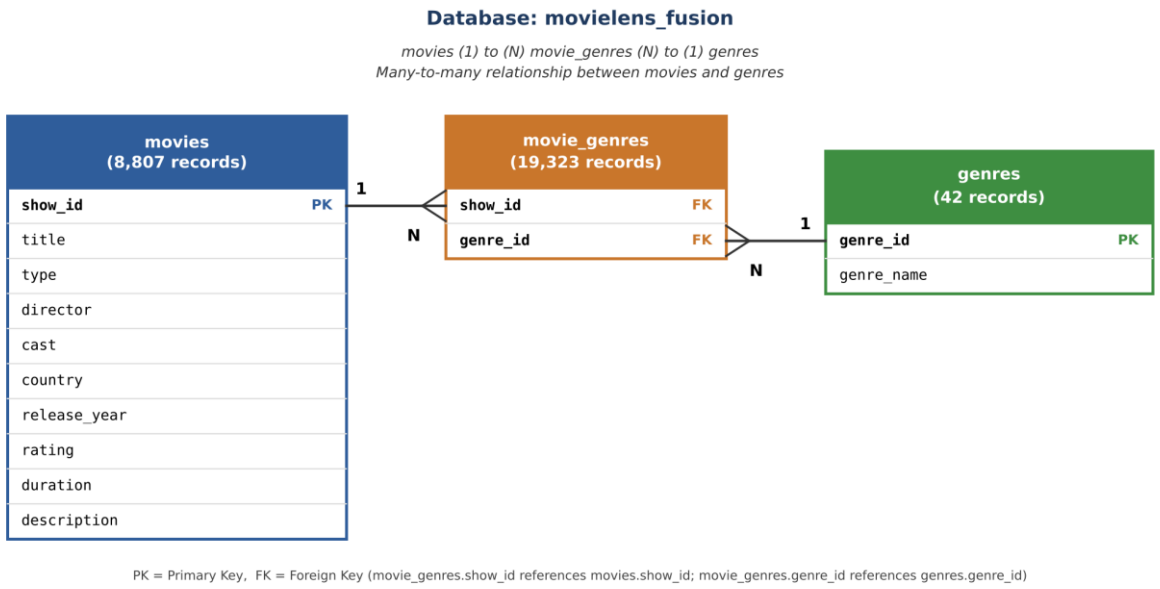


Figure 5.6: Database Relationship (ER) Diagram

The movies table stores the descriptive attributes of each title, namely show_id, title, type, director, cast, country, release_year, rating, duration and description. The genres of a title are not stored in the movies table; they are held in the genres and movie_genres tables, which is what makes the many-to-many relationship possible.

5.4 Frontend and Backend Communication

Figure 5.3 shows the request and response cycle between the layers. The frontend never connects to MySQL directly. It sends an HTTP request to a Flask route, Flask executes the SQL query, and the result returns through Flask as a response that React renders in the interface.

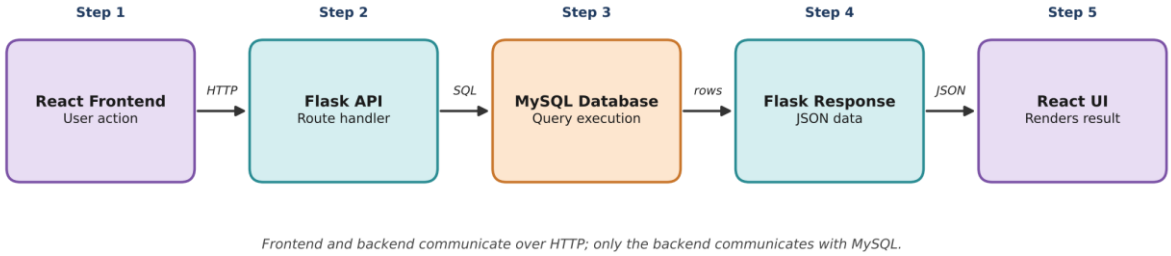


Figure 5.7: Frontend and Backend Communication

CHAPTER 6
Implementation

This chapter explains how each part of the pipeline and application is implemented, from data ingestion to the web application.

6.1 Module-wise Implementation

Table 6.1: Implementation modules

Module	Responsibility	Main tools
Data ingestion	Load the raw CSV into memory and review its size and columns	Python, Pandas
Data cleaning and preprocessing	Handle missing values, check duplicates, standardize values and data types	Python, Pandas
Data transformation	Select and filter columns, derive duration fields, aggregate records	PySpark
Genre extraction	Split the multi-valued genre field into individual genres	PySpark
Database loading	Create the schema and populate movies, genres and movie_genres	MySQL
Backend API	Query MySQL and return data to the frontend	Flask, Python
Frontend application	Display data, support search and show analytics	React, Vite, JavaScript, CSS

6.2 Data Ingestion

The dataset is a CSV file of movie and TV show records. It has 12 original columns: show_id, type, title, director, cast, country, date_added, release_year, rating, duration, listed_in and description. It contains 8,807 records, of which 6,131 are movies and 2,676 are TV shows. Ingestion reads this file into a Pandas DataFrame and reviews its shape, column names and data types before any change is made. The ingestion step confirms that the file holds 8,807 rows and 12 columns and that the type column divides the records into 6,131 movies and 2,676 TV shows.

6.3 Data Cleaning and Preprocessing

Cleaning is performed with Python and Pandas and proceeds in the following order: the CSV file is read and its structure and data types are inspected; missing values are identified column by column and handled; duplicate records are checked using both the show_id column and entire rows; values are standardized and data types are converted where required; and the cleaned dataset is written out for the next stage. Two derived columns, duration_value and

duration_unit, are created by separating the numeric part of the duration field from its unit (minutes for a movie or seasons for a TV show). This brings the cleaned dataset to 14 columns.

The cleaning report produced for the dataset is summarized in Table 6.2.

Table 6.2: Actual data cleaning report

Cleaning Measure	Result
Records processed	8,807
Rejected records	0
Duplicate IDs	0
Duplicate rows	0
Columns in raw dataset	12
Columns in cleaned dataset	14

6.4 Data Transformation Using PySpark

After cleaning, PySpark is used for transformation. The processing steps are: reading the processed data, inspecting the schema, filtering and selecting columns, transforming duration information, extracting genre information, grouping and aggregating records, and writing the processed data to Parquet files. Working with Spark DataFrames means that each step is expressed as a transformation on a distributed table, and the final write stores the result in a columnar file format.

The processed data is stored in Parquet format. Parquet keeps the schema with the data, stores values column by column and compresses well, which makes later reads efficient. It also gives the pipeline a clear intermediate stage between PySpark processing and the MySQL load.

6.5 Genre Extraction and Processing

Each title in the dataset has a listed_in field containing its genre or genres. A relational database cannot store several genres in one column properly, so the genre information is extracted into separate values. Extraction produces a set of 42 distinct genres and 19,323 links between titles and genres, an average of about 2.19 genres per title (19,323 divided by 8,807).

The extracted values become two tables. The genres table stores each distinct genre once and gives it a genre_id. The movie_genres table stores one row for every pairing of a title (show_id) with one of its genres (genre_id). In this way a title such as one belonging to three genres appears in the movies table once and in the movie_genres table three times.

6.6 MySQL Database Implementation

The processed data is stored in a MySQL database named movielens_fusion. Table 6.3 lists its tables and record counts.

Table 6.3: Tables of the movielens_fusion database

Table	Records	Description
movies	8,807	One row per movie or TV show; primary key show_id
genres	42	One row per distinct genre; primary key genre_id
movie_genres	19,323	Links titles to genres using show_id and genre_id

The three tables are connected as follows. movies.show_id is the primary key of the movies table and genres.genre_id is the primary key of the genres table. In movie_genres, show_id is a foreign key that refers to movies.show_id, and genre_id is a foreign key that refers to genres.genre_id. To find the genres of a title, or the titles of a genre, the database joins movies, movie_genres and genres.

The database movielens_fusion is created on the MySQL server running on the host machine, and its tables are populated from the processed data produced by the PySpark stage. After loading, the record count of each table is compared with the expected count from the processed dataset (Table 8.1): 8,807 titles, 42 genres and 19,323 title-genre links. The backend connects to this database to answer API requests, and its health-check endpoint repeats the connectivity and record-count check while the application is running.

6.7 Backend Implementation Using Flask

The backend is written in Python with the Flask framework. When the Flask application starts, it connects to the movielens_fusion database on MySQL, enables CORS for the browser-based frontend and registers its route modules as Flask Blueprints. Each endpoint is a route function that runs when a particular URL is requested, executes a query and returns the data to the caller as a JSON response. In the Docker deployment the application is run by Gunicorn and listens on port 5000 inside the container.

The application registers six route modules, listed in Table 6.4. The movies, genres and analytics modules supply the data used by the exploration, genre analysis and dashboard features. The recommendations, auth and user_activity modules are registered with the application, but the features of this report do not rely on them, and a registered module is not treated here as a completed feature.

Table 6.4: Route modules registered by the Flask application

Blueprint Module	Area	Role in the Application
movies	Movies and TV shows	Retrieves movie and TV show records for listing, search, filtering and detail views.
genres	Genres	Provides genre information from the genres and movie_genres tables.
analytics	Analytics	Provides the summary and aggregated data used by the dashboard and the genre-based analysis.
recommendations	Recommendations	Registered with the application. Recommendation functionality is not part of the verified scope of this report; planned improvements are described in Chapter 11.
auth	Authentication	Registered with the application. It is not part of the verified scope of this report.
user_activity	User activity	Registered with the application. It is not part of the verified scope of this report.

In addition to the data routes, the backend provides a health-check endpoint at /api/health. When it is called, the backend tests the connection to the MySQL database and reads the record counts of the tables. The endpoint therefore confirms in a single request that the service is running, that the database is reachable and that the data has been loaded. Its response is returned as JSON like the other routes.

6.8 Frontend Implementation Using React

The frontend is a React application built with Vite, written in JavaScript and styled with CSS. React divides the interface into components, each responsible for one part of the page, and re-renders them when data changes. The application retrieves its data by calling the Flask API over HTTP from the browser and presents the responses as lists, search results, genre views and analytics.

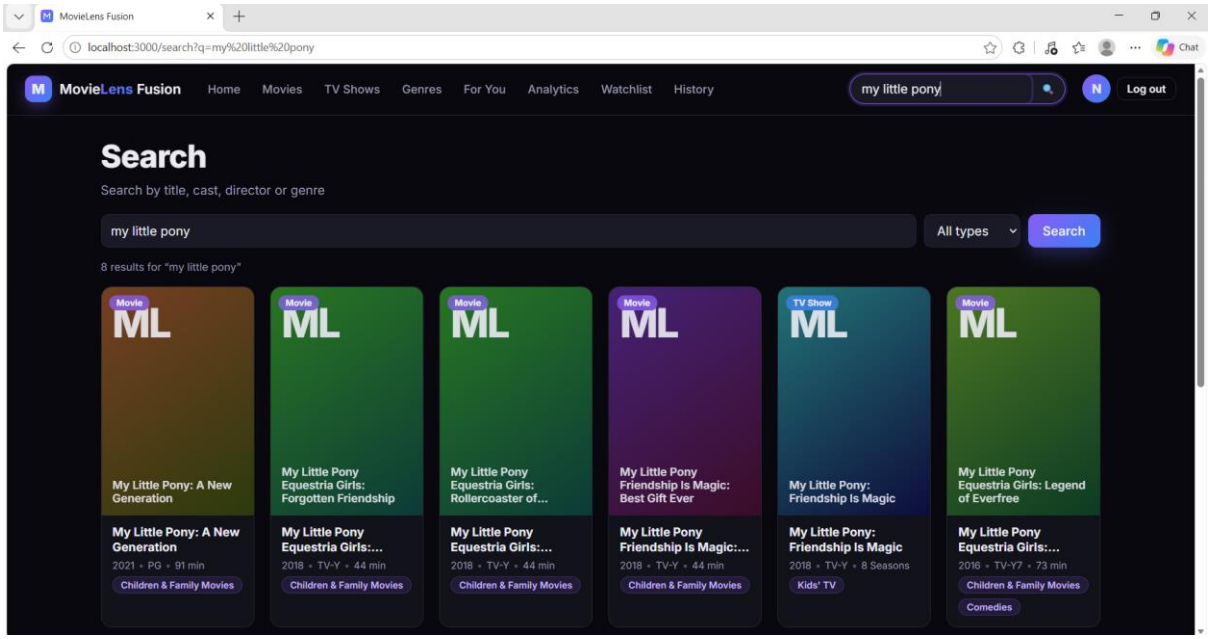
The frontend provides the dashboard, search and filtering of titles, information about movies and TV shows, genre-based analysis and analytics charts described in Chapter 7. It never connects to MySQL directly; every piece of data it shows is returned by the Flask API. In the Docker deployment the built application is served by Nginx on port 3000.

CHAPTER 7
Features

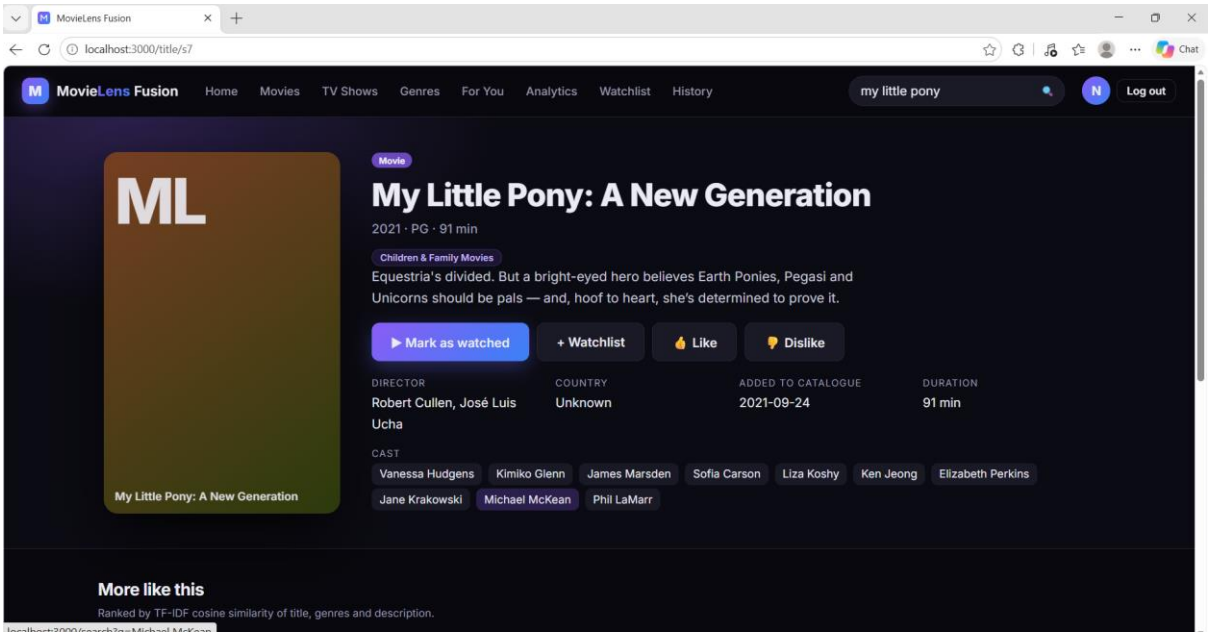
7.1 List of Main Features

The main features of the MovieLens Fusion application and pipeline are:

- Cleaning and quality reporting of a movie and TV show dataset of 8,807 records.
- Genre extraction that links each title to one or more of 42 genres.
- Storage of processed data in Parquet files and in a relational MySQL database.
- Exploration and search of movie and TV show information.
- Analysis of movies and TV shows.
- Genre-based analysis.
- An interactive analytics dashboard.
- Containerized deployment of the frontend and backend using Docker Compose.



- **Figure 7.1:** Movie Search Results



- **Figure 7.2:** Movie and TV Show Details

7.2 Data Exploration and Search

Users can explore the records stored in the movies table and search for titles. Because the data comes from MySQL through the Flask backend, search and filter requests are answered from the database rather than from a file loaded into the browser. Typical attributes available for exploration are the title, type, director, cast, country, release year, rating, duration and description of each record.

7.3 Movie and TV Show Analysis

The dataset contains two content types. Of the 8,807 records, 6,131 are movies (69.6 percent) and 2,676 are TV shows (30.4 percent). The cleaning step separates the duration field into duration_value and duration_unit so that movies, measured in minutes, and TV shows, measured in seasons, can be analyzed consistently. Figure 7.1 shows the composition of the dataset.

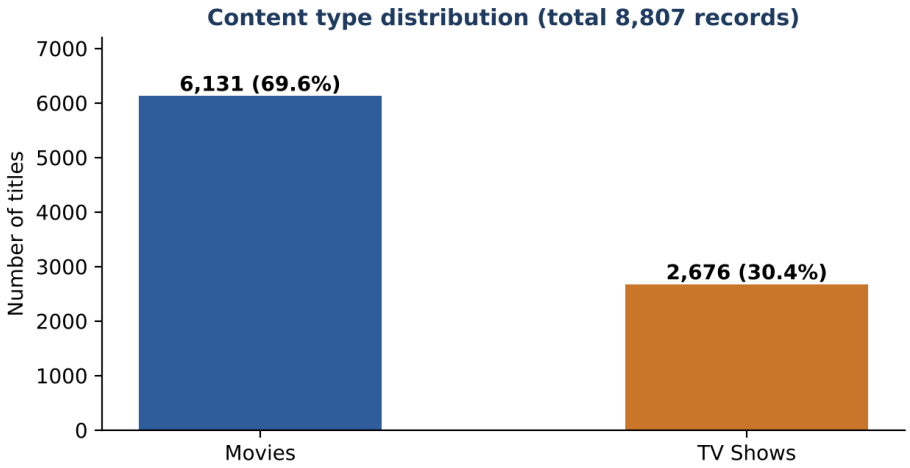


Figure 7.3: Content Type Distribution of the Dataset

7.4 Genre-based Analysis

Genre information is stored in the genres and movie_genres tables, which allows titles to be filtered and counted by genre. Since a title can have several genres, it is counted under each of them. The 42 genres and 19,323 title-genre links form the basis of the genre analytics.

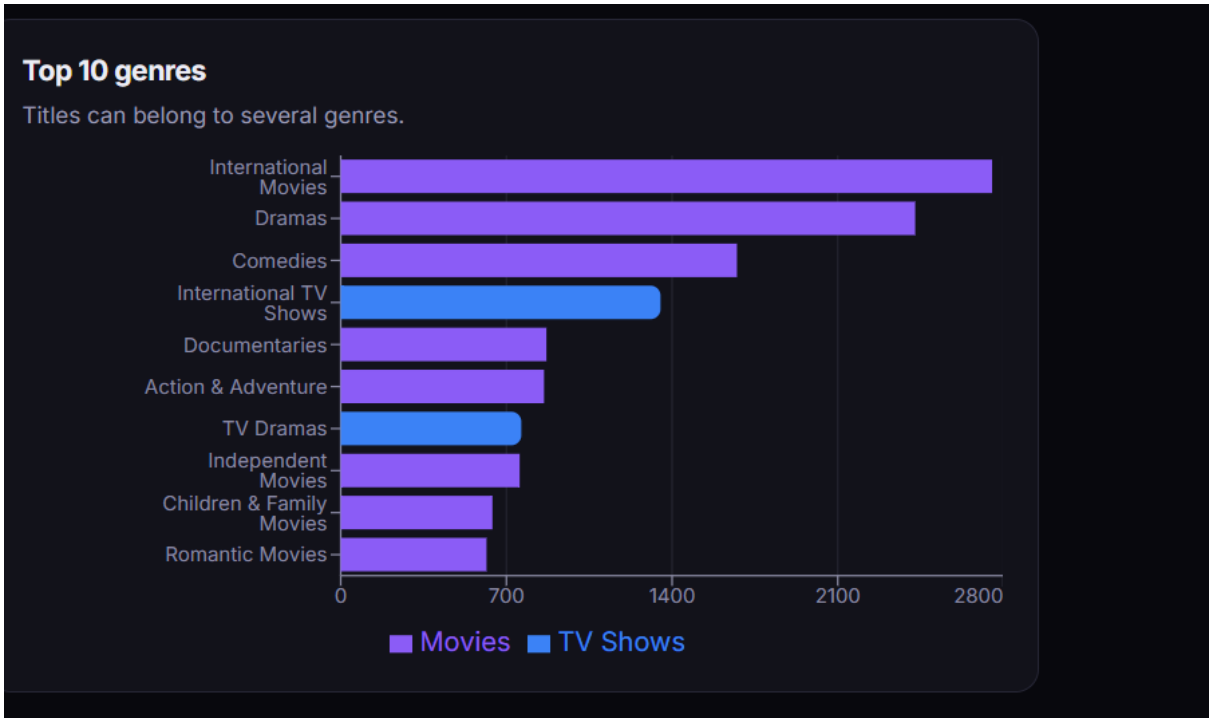


Figure 7.4: Genre-based Analysis

7.5 Interactive Analytics Dashboard

The frontend presents analytics through an interactive dashboard so that users can view summaries of the data without writing queries. The dashboard is built with React and receives its data from the Flask API.

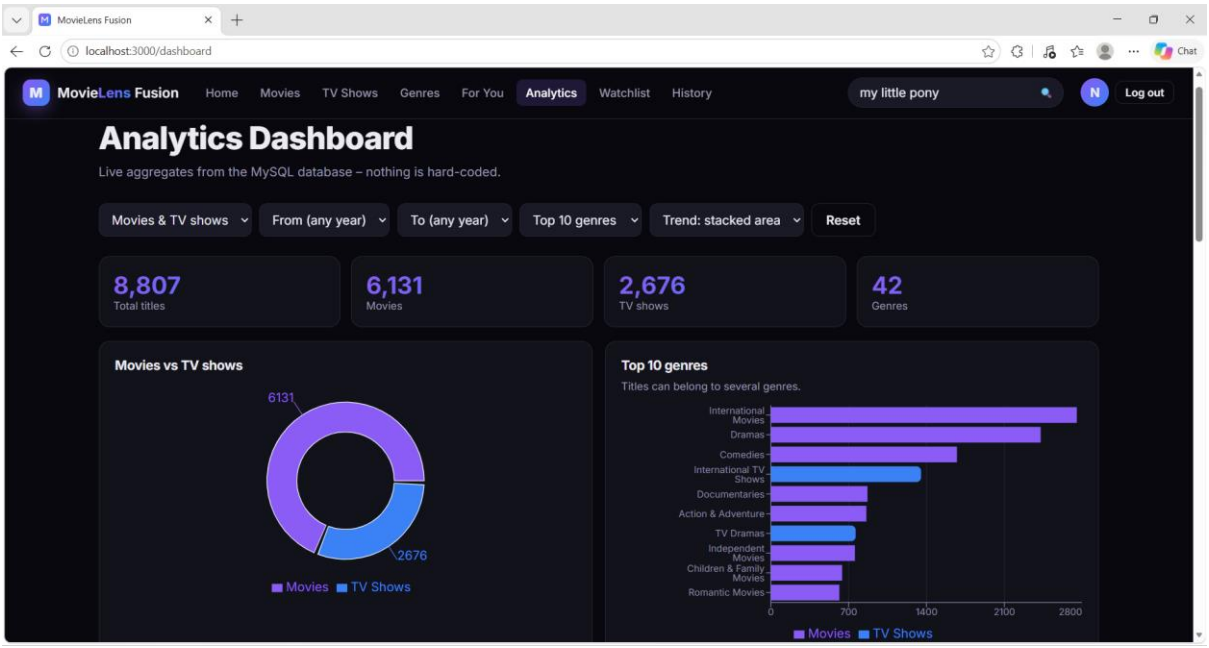


Figure 7.5: Analytics Charts on the Dashboard

CHAPTER 8
Testing

This chapter reports the testing and validation of the project. The results recorded as passed are those established from the processing outputs of the project: the dataset statistics, the cleaning report and the MySQL table counts. No processing times, accuracy percentages or response times are reported.

8.1 Functional Testing

Functional testing checks whether each function of the system produces the expected result. For the data pipeline the relevant checks are loading the dataset, counting the records, identifying duplicates and confirming the structure of the cleaned data (Table 8.2). For the web application the relevant functions are loading the pages, searching, filtering and viewing analytics; the way in which they are verified is described in Section 8.6.

8.2 Integration Testing

Integration testing checks that the layers work together: the pipeline output reaches MySQL, the Flask backend can read from MySQL, the React frontend can call the Flask API, and the Docker containers can communicate with each other and with the host database. The first of these, the transfer of the processed data into MySQL, is validated by the record counts in Table 8.1 and Table 8.2. The remaining checks are described in Section 8.6.

8.3 Database Testing

Database testing compares the contents of the MySQL tables with the processed dataset. The record counts of the three tables are shown in Table 8.1 and agree with the dataset statistics: the movies table has the same 8,807 records as the cleaned dataset, the genres table holds 42 distinct genres, and movie_genres holds 19,323 title-genre links.

Table 8.1: MySQL record counts

Table	Expected Records	Records in MySQL
movies	8,807	8,807
genres	42	42
movie_genres	19,323	19,323

8.4 Test Cases and Results

Table 8.2: Test cases and results

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	Load the raw CSV dataset	8,807 records processed	8,807 records processed	Passed
TC-02	Count movies and TV shows	6,131 movies and 2,676 TV shows (total 8,807)	6,131 and 2,676; sum is 8,807	Passed
TC-03	Check duplicate IDs	0 duplicate IDs	0 duplicate IDs	Passed
TC-04	Check duplicate rows	0 duplicate rows	0 duplicate rows	Passed
TC-05	Check rejected records	0 rejected records	0 rejected records	Passed
TC-06	Check cleaned dataset structure	14 columns	14 columns	Passed
TC-07	MySQL movies table count	8,807 records	8,807 records	Passed
TC-08	MySQL genres table count	42 records	42 records	Passed
TC-09	MySQL movie_genres table count	19,323 records	19,323 records	Passed

8.5 Data Quality Evaluation

Data quality is evaluated from the cleaning report and the database counts. No records were rejected, and no duplicate IDs or duplicate rows were found. The cleaned dataset has two more columns than the raw data because the duration field was separated into duration_value and duration_unit.

Table 8.3: Dataset statistics and data quality summary

Measure	Value
Records processed	8,807
Rejected records	0
Duplicate IDs	0
Duplicate rows	0
Movies	6,131
TV shows	2,676
Columns (raw / cleaned)	12 / 14
Distinct genres	42
Title-genre links	19,323
Average genres per title (derived: 19,323 / 8,807)	approx. 2.19

These figures describe the structure and consistency of the data. They do not measure accuracy against an external source, which was not part of this project.

8.6 Application and Deployment Verification

The application layers are verified by running them together and checking the following points:

- Backend health: the /api/health endpoint tests the connection to the MySQL database and returns the record counts of the tables. A correct response shows that the database is reachable and that the counts are 8,807 for movies, 42 for genres and 19,323 for movie_genres.
- Backend API: each data route registered for movies, genres and analytics returns a JSON response built from a MySQL query.
- Parquet output: the PySpark stage writes the Parquet files from which the processed data is loaded into MySQL.
- Frontend: the React application loads in the browser and displays data retrieved from the API, and the search, filtering, genre analysis and dashboard views show results that agree with the database.
- Docker deployment: docker compose up --build builds the two images and starts the frontend, which is reachable on port 3000, and the backend, which queries the MySQL database on the host machine.

Table 8.2 records the results established from the processing outputs. The points above are the acceptance criteria applied to the running application and its Docker deployment.

CHAPTER 9
Deployment

9.1 Docker Configuration

The application is deployed with Docker and Docker Compose. Figure 9.1 shows the deployment architecture. Two containers are defined. The frontend container builds the React application with Node.js and Vite in a build stage and serves the result with Nginx on port 3000. The backend container runs the Flask application under Gunicorn on port 5000 inside the container. MySQL is not a Compose service: the backend connects to the existing MySQL server running on the host machine.

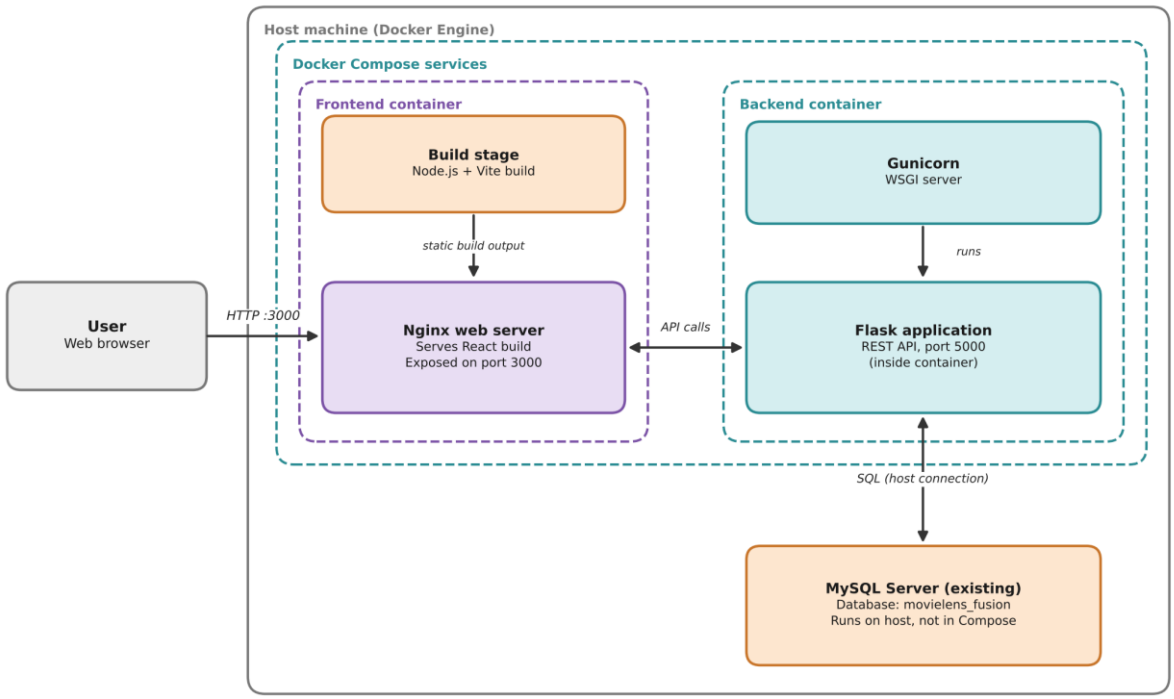


Figure 9.1: Docker Deployment Architecture

Table 9.1: Deployment components

Component	Role	Port
Frontend container	Builds the React application with Node.js and serves it with Nginx	3000 (exposed)
Backend container	Runs the Flask application with Gunicorn	5000 (inside container)
MySQL server	Existing database movielens_fusion on the host machine; not a Compose service	Host machine (Windows)

9.2 Backend Containerization

The backend image is based on Python. The Dockerfile installs the Python dependencies, copies the Flask application and starts it with Gunicorn, which is a production-grade WSGI server and is preferred over the Flask development server for running containers. The application listens on port 5000 inside the container.

9.3 Frontend Containerization

The frontend image is built in two stages. In the first stage, Node.js installs the dependencies and Vite produces the optimized production build. In the second stage, Nginx serves the built static files. Only the Nginx stage is needed at run time, which keeps the final image smaller than an image that contains the Node.js toolchain. The application is exposed on port 3000.

9.4 Docker Compose Execution

Docker Compose starts the frontend and backend together with a single command. Because MySQL runs outside Compose, the MySQL server must be running on the host machine, with the database movielens_fusion loaded, before the services are started.

The standard Docker Compose commands used to build, inspect and stop the application are:

```
docker compose up --build     # build images and start the services
docker compose ps               # list running containers
docker compose logs                  # view service logs
docker compose down            # stop and remove the containers
```

Docker Compose places the frontend and backend containers on a common network, so the two services can communicate with each other. The frontend container publishes port 3000 to the host so that the application can be opened in a browser at http://localhost:3000, while port 5000 of the backend is used inside the container.

MySQL is not a Compose service; it runs directly on the Windows host machine. Inside a container, the name localhost refers to the container itself and not to the host, so the backend must be configured with an address through which the container can reach the host, together with the user name, password and database name movielens_fusion. Keeping MySQL outside Compose means that the existing database and its data are used unchanged, and it is the reason the database server must be started before docker compose up.

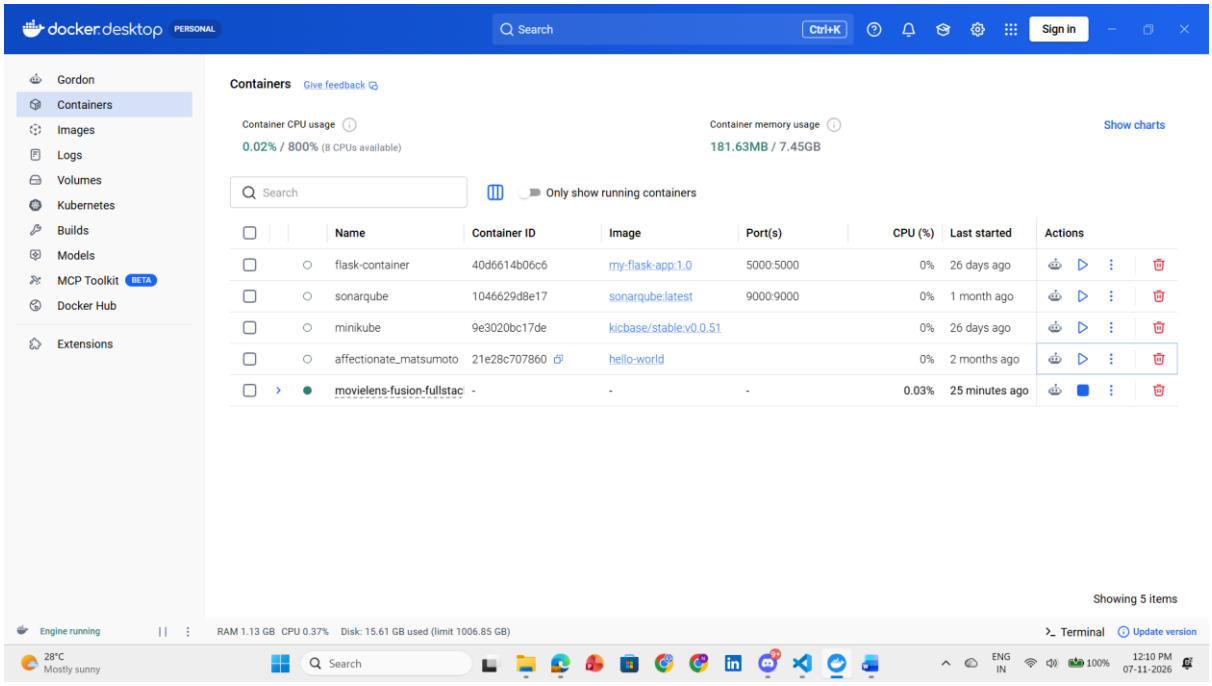


Figure 9.2: Docker Containers Running Successfully

9.5 Local Application Execution

The application can also be run without Docker. The database is created and loaded in MySQL, the Flask backend is started in a Python virtual environment, and the React application is started with the Vite development server. The detailed steps are given in Section 14.2. Running locally is useful during development because code changes appear immediately, while the Docker deployment is used to run the application in a repeatable environment.

CHAPTER 10
Challenges and Limitations

10.1 Issues Faced During Development

The development of a pipeline that connects several technologies raised challenges at each stage. The main ones are described below.

Missing values in the original dataset

Fields such as director, cast, country and rating are not filled for every title in datasets of this kind. Missing values had to be identified and handled before the data could be processed reliably.

Inconsistent data formats

The duration field combines a number and a unit, and the genre field combines several values in one cell. These formats could not be analyzed or stored relationally without restructuring.

PySpark environment configuration

PySpark depends on a compatible Java installation and a correctly configured Python environment, which makes the initial setup more demanding than for a plain Python script.

Data transformation and genre extraction

Turning a multi-valued text column into a normalized genres table and a movie_genres link table required careful splitting, trimming and de-duplication of genre names.

MySQL connection configuration

The backend needs correct host, user, password and database settings, and the database must be reachable from the application, including from inside a container.

Flask API integration

The backend routes had to return data in a form the frontend can consume and had to match the structure of the database tables.

React frontend and backend communication

The frontend and backend run as separate applications on different ports, so requests from the browser to the API must be configured to work correctly. Cross-Origin Resource Sharing (CORS) is enabled in the Flask backend so that the browser accepts the responses of the API.

Docker configuration and container networking

Because MySQL runs on the host rather than in Compose, the backend container has to reach a service outside the Docker network.

10.2 Solutions Applied

Table 10.1 relates each issue to the approach used in the project.

Table 10.1: Issues and solutions

Issue	Approach in the Project
Missing values	Missing values were identified and handled in the Pandas cleaning stage. The cleaning report shows 8,807 records processed and 0 rejected.
Inconsistent formats	The duration field was separated into duration_value and duration_unit, giving a cleaned dataset of 14 columns.
Genre extraction	The multi-valued genre field was split with PySpark into the genres table (42 genres) and the movie_genres table (19,323 links).
PySpark and Java setup	A compatible Java Development Kit and a configured Python environment were installed so that PySpark could run the transformation and write the Parquet files.
MySQL connection	The backend is configured with the host, user, password and database name (movielens_fusion) of the MySQL server. Inside a container the host machine must be addressed explicitly, since localhost refers to the container.
Flask API integration	The routes are organized as Blueprint modules that query MySQL and return JSON responses; /api/health checks database connectivity and record counts.
Frontend and backend communication	The React frontend calls the Flask API over HTTP, and CORS is enabled in the backend so that browser requests from the frontend are accepted.
Container networking	The Compose configuration defines two containers, a frontend served by Nginx on port 3000 and a backend run by Gunicorn on port 5000. The backend connects to the MySQL server on the Windows host machine.

10.3 Current Limitations

- The system uses a single movie and TV show dataset; data from other sources is not integrated.
- The application provides data exploration and analytics, but it does not provide a completed recommendation feature. Route modules for recommendations, authentication and user activity are registered in the backend, but they are outside the verified scope of this report, and recommendations are treated as a future enhancement.
- The pipeline is a batch process: new or changed source data must be run through the pipeline again before it appears in the application.
- MySQL is not containerized in the current Compose configuration, so the Docker deployment depends on a database that is already installed and running on the host machine.
- The application is deployed locally with Docker; it is not deployed on a cloud platform.

- For a dataset of 8,807 records PySpark is not strictly necessary; its benefit would appear with much larger data.
- The results recorded in Chapter 8 are limited to the dataset statistics and the database record counts; no processing-time or response-time measurements are reported.
- The dataset has no language column, and it contains no user rating data, so the application offers no language-based or rating-based analysis.

CHAPTER 11
Future Enhancements

The following enhancements are possible extensions of the project. They are proposals only and are not part of the current implementation.

11.1 Multi-source Data Integration

The pipeline could be extended to combine several movie datasets, for example by adding data from public movie databases such as IMDb or TMDB. This would require matching titles across sources, resolving conflicting values and extending the schema. Integration of this kind would also increase the importance of the data quality checks already used in the cleaning stage.

11.2 Advanced Recommendation Systems

The genre structure already stored in MySQL could be used as input for recommendation methods. Content-based methods could suggest titles with similar genres, and collaborative or hybrid methods could be added if rating data from users becomes available. More analytics and visualizations could be added alongside these. The dataset has no language column, so recommendations based on language are not possible with the current data and would require an additional data source.

11.3 User Preference-based Recommendations

A later version could let users state or record preferences, such as favorite genres, and use them to personalize the titles that are shown. This would require storing user preference data and adding the corresponding backend endpoints and frontend screens.

11.4 Cloud Deployment

The Docker containers could be deployed on a cloud platform, and the MySQL database could be moved to a managed database service. Cloud deployment would make the application available to users outside the local machine and would improve scalability, since the processing and serving components could be scaled independently.

CHAPTER 12
Conclusion

12.1 Summary of the Project

MovieLens Fusion: An Integrated Movie Analytics Pipeline is an end-to-end project that converts a raw movie and TV show CSV dataset into a working web application. The dataset contains 8,807 records, of which 6,131 are movies and 2,676 are TV shows.

The data engineering pipeline begins with Python and Pandas, which inspect and clean the data. The cleaning report shows 8,807 records processed with 0 rejected records, 0 duplicate IDs and 0 duplicate rows. PySpark then transforms the data, including duration processing and genre extraction, and the results are stored as Parquet files. The processed data is loaded into the MySQL database movielens_fusion, which has three tables: movies (8,807 records), genres (42 records) and movie_genres (19,323 records), connected by a many-to-many relationship.

A Flask backend exposes the stored data through API endpoints, and a React and Vite frontend lets users explore and analyze it. The backend and frontend are containerized with Docker and run together using Docker Compose, with the backend connecting to the existing MySQL database on the host machine.

The verified results of the project are the dataset statistics (8,807 records, 6,131 movies, 2,676 TV shows, 12 raw and 14 cleaned columns, 0 rejected records) and the database counts (8,807, 42 and 19,323). The current implementation uses a single dataset, runs MySQL on the host machine outside Docker Compose, and treats recommendations as future work.

12.2 Achievements

- Built a complete pipeline from raw CSV data to a user-facing web application.
- Cleaned and checked a dataset of 8,807 records with no rejected records and no duplicates.
- Used PySpark for transformation and genre extraction and stored the output in Parquet format.
- Designed and populated a relational MySQL database with three related tables.
- Developed a Flask backend and a React and Vite frontend that work together.
- Containerized the application with Docker and Docker Compose and exposed it on port 3000.

12.3 Skills Learned During Development

- Data inspection and cleaning with Python and Pandas.
- Distributed data processing with PySpark.
- Working with the Parquet storage format.
- Relational database design, many-to-many relationships and SQL.
- Building REST-style APIs with Flask.
- Developing interactive interfaces with React and Vite.
- Containerization with Docker, Gunicorn, Nginx and Docker Compose.
- Team collaboration and technical documentation.

CHAPTER 13
References

Research Papers

[1] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," Communications of the ACM, vol. 59, no. 11, pp. 56–65, 2016. doi:10.1145/2934664.

[2] M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2012.

[3] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conference, 2010, pp. 56–61.

[4] E. Rahm and H. H. Do, "Data Cleaning: Problems and Current Approaches," IEEE Data Engineering Bulletin, vol. 23, no. 4, pp. 3–13, 2000.

[5] R. Y. Wang and D. M. Strong, "Beyond Accuracy: What Data Quality Means to Data Consumers," Journal of Management Information Systems, vol. 12, no. 4, pp. 5–33, 1996.

[6] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970. doi:10.1145/362384.362685.

Dataset References

[7] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," ACM Transactions on Interactive Intelligent Systems, vol. 5, no. 4, Article 19, 2015. doi:10.1145/2827872.

[8] Movie and TV show catalogue dataset (CSV file, 8,807 records and 12 columns), used as the input dataset of this project.

Technical Documentation

[9] Python Software Foundation, Python Documentation. Available: https://docs.python.org/3/

[10] pandas Development Team, pandas Documentation. Available: https://pandas.pydata.org/docs/

[11] The Apache Software Foundation, PySpark Documentation. Available: https://spark.apache.org/docs/latest/api/python/

[12] The Apache Software Foundation, Apache Parquet Documentation. Available: https://parquet.apache.org/docs/

[13] Oracle Corporation, MySQL Reference Manual. Available: https://dev.mysql.com/doc/

[14] Pallets Projects, Flask Documentation. Available: https://flask.palletsprojects.com/

[15] Meta Open Source, React Documentation. Available: https://react.dev/

[16] Vite Team, Vite Documentation. Available: https://vite.dev/guide/

[17] Docker Inc., Docker Documentation. Available: https://docs.docker.com/

[18] Docker Inc., Docker Compose Documentation. Available: https://docs.docker.com/compose/

[19] Gunicorn Documentation. Available: https://docs.gunicorn.org/

[20] Nginx Documentation. Available: https://nginx.org/en/docs/

CHAPTER 14
Appendices

14.1 Project Components and Configuration Files

Table 14.1 lists the main components of the project that make up the implementation described in Chapters 6 and 9.

Table 14.1: Main project components

Component	Technology	Description
Data cleaning script	Python, Pandas	Reads the raw CSV file, handles missing values, checks duplicates and creates duration_value and duration_unit (14 columns).
PySpark processing script	PySpark, Parquet	Transforms the cleaned data, extracts genres and writes the processed data to Parquet files.
SQL schema and loading	MySQL	Creates the database movielens_fusion and the tables movies, genres and movie_genres, and loads the processed data.
Flask application	Flask, Python	Initializes the application, connects to MySQL, enables CORS, registers the route modules and provides /api/health.
Flask route modules	Flask Blueprints	movies, genres, analytics, recommendations, auth and user_activity.
React frontend	React, Vite, CSS	Dashboard, search and filtering, movie and TV show information, genre analysis and analytics.
Backend Dockerfile	Docker, Gunicorn	Builds the backend image that runs the Flask application on port 5000.
Frontend Dockerfile and Nginx configuration	Docker, Node.js, Nginx	Builds the React application and serves it with Nginx on port 3000.
docker-compose.yml	Docker Compose	Defines the frontend and backend services; MySQL is not included.

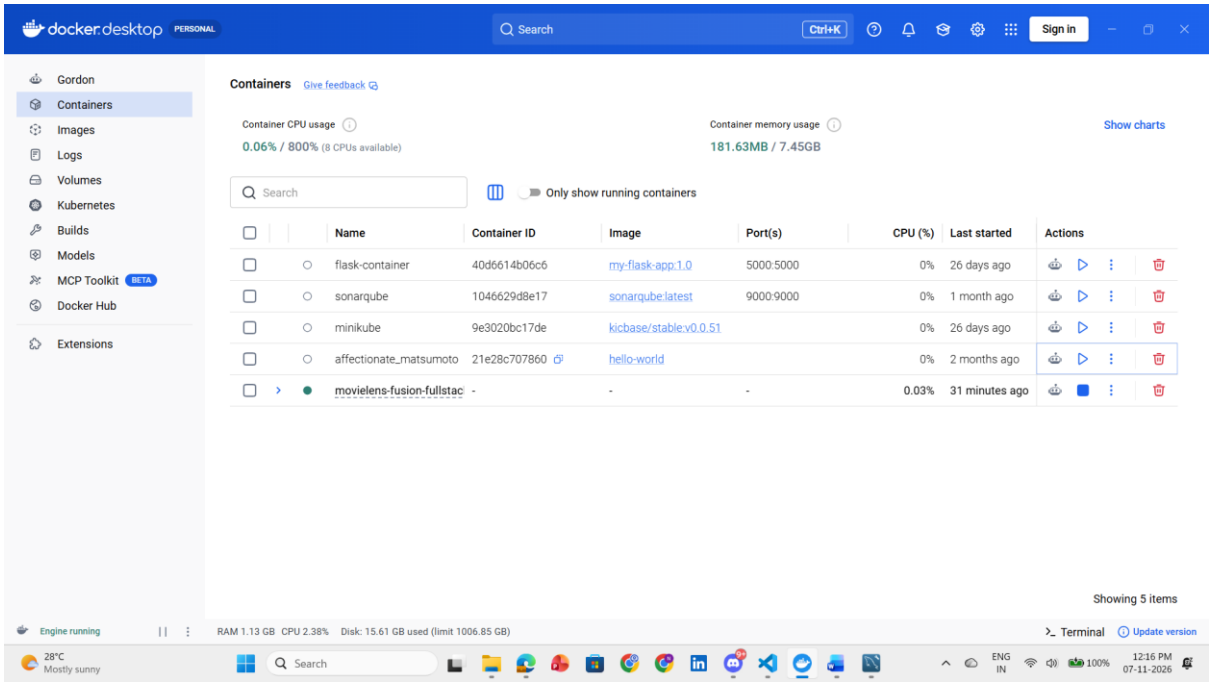


Figure 14.1: Additional Application Screenshot

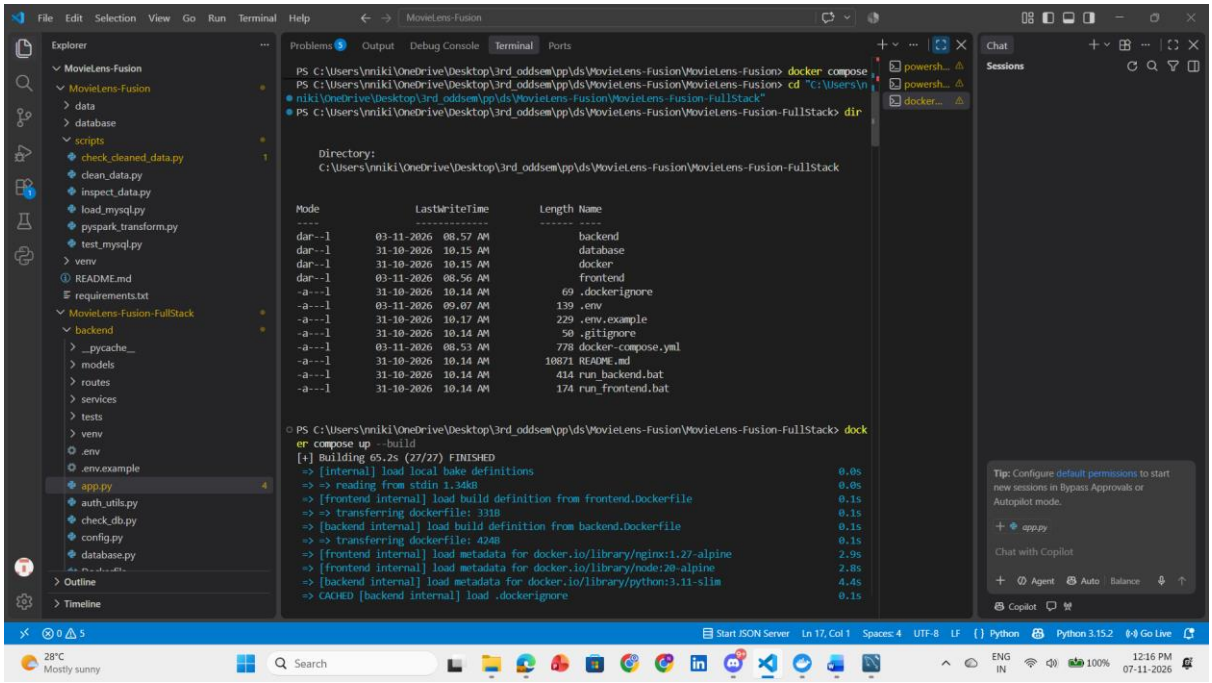


Figure 14.2: Terminal Execution Screenshot

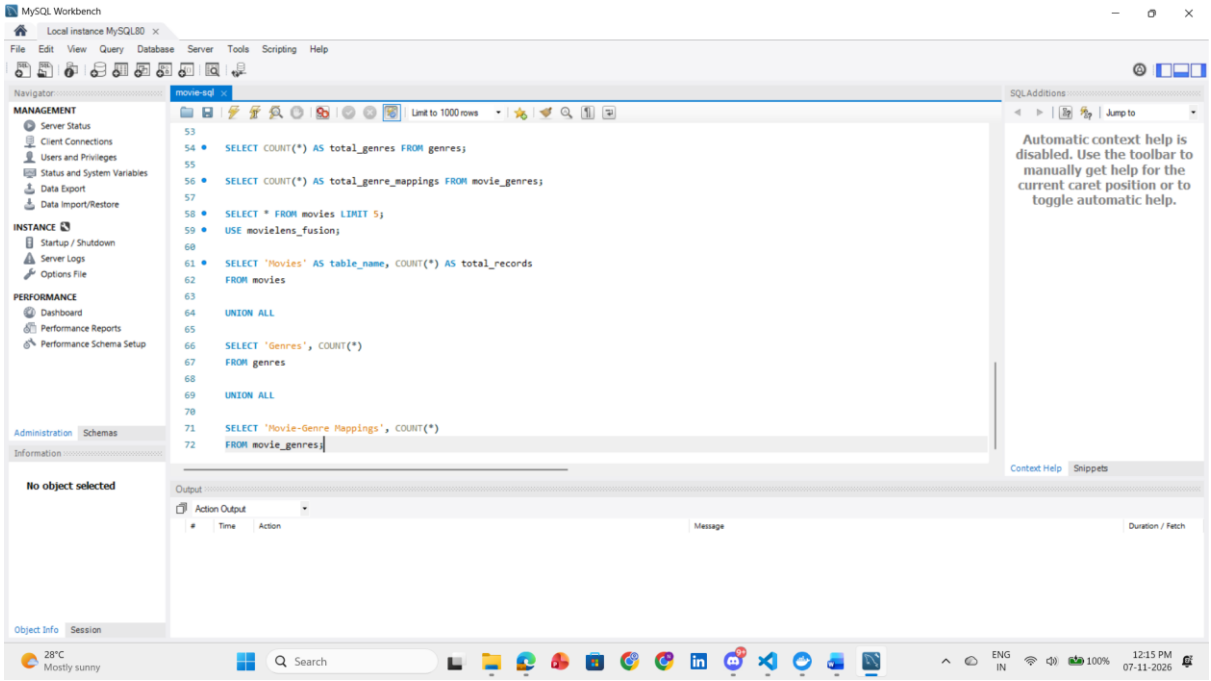


Figure 14.3: MySQL Database Screenshot

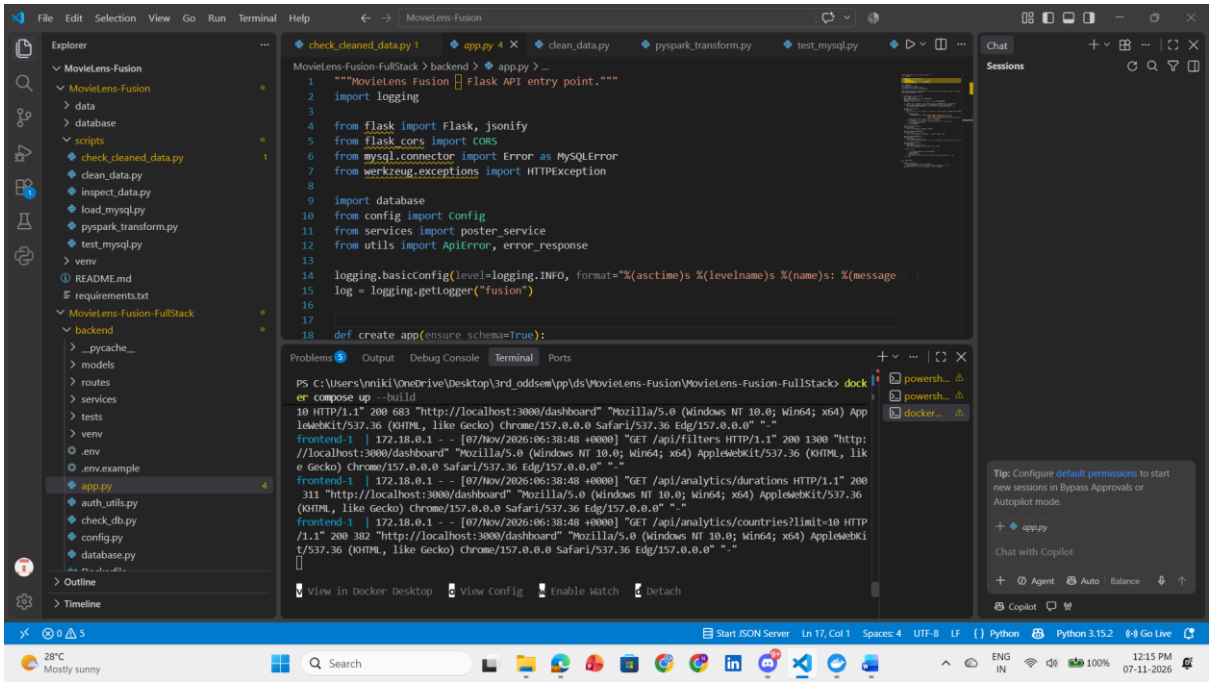


Figure 14.4: Project Folder Structure

14.2 Installation and Setup Instructions

The steps below describe how the project is set up and run on a Windows machine, on which the MySQL server runs in both the local and the Docker execution modes.

Step 1: Install the required software

Install Python 3.x, a Java JDK, MySQL Server, Node.js with npm, and Docker Desktop (or Docker Engine with Docker Compose).

Step 2: Set up the Python environment

```
python -m venv venv
venv\Scripts\activate
```

Step 3: Install backend dependencies

Install the backend dependencies in the active virtual environment with pip, using the dependency list of the backend.

Step 4: Configure MySQL

```
mysql -u root -p
CREATE DATABASE movielens_fusion;
```

Then load the movies, genres and movie_genres tables from the processed data.

Step 5: Configure environment variables

Provide the database connection details to the backend: the host address, user name, password and the database name movielens_fusion.

Step 6: Install frontend dependencies

```
cd frontend
npm install
```

Step 7: Run the application locally

Start the Flask backend, and then start the React frontend with the Vite development server:

```
npm run dev
```

The Vite development server prints the local address of the application in the terminal.

Step 8: Run the application with Docker Compose

```
docker compose up --build
```

Make sure the MySQL server is running on the host machine first. When the containers are running, open http://localhost:3000 in a browser. To stop the application run docker compose down.

14.3 User Manual

The following steps describe how a user works with the application.

1. Open the application in a web browser (for the Docker deployment: http://localhost:3000).
2. Review the home page or dashboard to see the overview of the data.
3. Explore the movie and TV show records, including their type, release year, rating and duration.
4. Use the search and filter options to find titles that match a name, type or genre.
5. Open a title to read its details such as director, cast, country and description.
6. Open the genre analysis to see how titles are distributed across genres.
7. Use the analytics charts on the dashboard to view summaries of the dataset.

14.4 Team Contributions

The project was developed jointly by the four team members, who collaborated on the data pipeline, the database, the backend, the frontend, the Docker deployment and the preparation of this report.

Table 14.2: Team members and contributions

Name	Roll Number	Responsibilities
DIKSHA YADAV	2420030001	Dataset collection, understanding, and initial data analysis.
K. BHAVANA	2420030332	Data cleaning using Pandas and data preprocessing.
G. TEJASHREE	2420030430	PySpark processing, genre extraction, and MySQL database management.
N. NIKITHA REDDY	2420030471	Flask backend development, React frontend integration, and Docker deployment.